SISTEMAS OPERACIONAIS — GUIA DE REVISÃO

Gerenciamento e Alocação de Memória

Apostila Completa: Da Monoprogramação à Paginação e Memória Virtual

1. ARQUITETURAS DE EXECUÇÃO DE SISTEMAS

O gerenciamento de memória evoluiu diretamente em resposta às arquiteturas de processamento de sistemas operacionais. A forma como os processos compartilham a CPU dita a complexidade do controle da memória RAM:

Sistemas Monoprogramáveis (Monotarefa)

Apenas um processo é executado por vez. Toda a CPU, memória RAM e periféricos ficam dedicados exclusivamente ao processo ativo até sua conclusão.

- **Vantagens:** Simplicidade absoluta; ausência de condição de corrida e sem necessidade de proteção de memória entre processos.
- **Desvantagem:** Extreme subutilização da CPU durante operações de Entrada/Saída (E/S), pois o processador fica ocioso aguardando discos ou periféricos.

Sistemas Multiprogramáveis (Multitarefa)

Múltiplos programas residem simultaneamente na RAM disputando a CPU, criando a ilusão de execução simultânea (pseudoparalelismo).

- **Funcionamento:** Quando o Processo A pausa para E/S, o SO imediatamente passa a CPU para o Processo B.
- **Desafios:** Exige mecanismos de proteção de memória, isolamento e sincronização para evitar *condições de corrida*.

2. ALOCAÇÃO PARTICIONADA (EVOLUÇÃO HISTÓRICA)

Antes do advento da memória virtual, os sistemas operacionais organizavam a memória RAM física dividindo-a em áreas conhecidas como partições contíguas:

Modelo	Mecanismo de Funcionamento	Impacto / Problema Principal
Partições Fixas (Estáticas)	A RAM é dividida na inicialização do sistema em blocos de tamanhos fixos (iguais ou desiguais). Os processos entram em filas para usar a partição.	**Fragmentação Interna:** Se um processo de 20 KB usa uma partição de 64 KB, os 44 KB restantes dentro do bloco ficam inacessíveis.
Partições Variáveis (Dinâmicas)	Não há partições pré-definidas. O SO cria uma partição sob medida na RAM com o tamanho exato solicitado pelo processo no momento de sua carga.	**Fragmentação Externa:** A saída de processos deixa "buracos" dispersos. A soma total de espaço livre pode ser grande, mas fragmentada.

Solução para Partições Variáveis: Compactação de Memória

Para reaproveitar os buracos dispersos da fragmentação externa, o SO desloca periodicamente todos os processos ativos para uma extremidade da RAM, unificando todos os espaços livres no outro extremo. Contudo, essa operação exige alto processamento da CPU e paralisa momentaneamente o sistema.

3. GERENCIAMENTO DE ESPAÇO LIVRE

Para controlar quais regiões da memória física estão ocupadas por processos e quais estão livres (lacunas ou buracos), o Sistema Operacional utiliza duas estruturas de dados fundamentais:

1. Mapa de Bits (Bitmap)

A memória é dividida em unidades fixas de alocação. Cada bloco corresponde a exatamente 1 bit no mapa (**1 = Ocupado**, **0 = Livre**).

- **Tamanho do Mapa:** Fixo e dependente do tamanho do bloco.
- **Desvantagem:** Busca lenta; encontrar um espaço de N blocos livres exige varrer o mapa bit a bit sequencialmente.

2. Lista Encadeada (Linked List)

Mantém uma lista ordenada de nós informando o tipo do segmento (**P** para Processo ou **L** para Lacuna), Endereço Inicial, Tamanho e Ponteiro.

- **Vantagem:** Agilidade na fusão de lacunas vizinhas quando um processo finaliza e libera memória.
- **Uso:** Base para os algoritmos de alocação dinâmica.

4. ALGORITMOS DE ALOCAÇÃO DE MEMÓRIA

Quando o SO utiliza listas encadeadas de espaços livres, ele aplica algoritmos para decidir em qual lacuna encaixar um novo processo que precisa entrar na RAM:

Algoritmo	Estratégia de Busca	Desempenho e Fragmentação
First Fit *(Primeiro Encaixe)*	Varre a lista a partir do início e aloca o processo no **primeiro buraco** que for grande o suficiente.	Rápido. Preserva blocos maiores no final da memória. É o mais eficiente na prática.
Next Fit *(Próximo Encaixe)*	Semelhante ao First Fit, mas inicia a busca a partir da **última posição alocada** na operação anterior.	Ligeiramente pior que o First Fit; tende a fragmentar os grandes blocos do final da memória.
Best Fit *(Melhor Encaixe)*	Varre toda a lista e escolhe o **menor buraco** capaz de abrigar o processo.	**Lento e crítico:** Produz centenas de micro-sobras de memória inavegáveis (altíssima fragmentação).
Worst Fit *(Pior Encaixe)*	Varre toda a lista e aloca o processo no **maior buraco** disponível.	Lento. Intenciona deixar sobras grandes, mas destrói rapidamente os maiores blocos livres disponíveis.

A Pegadinha Acadêmica do Best Fit

Embora intuitivamente o *Best Fit* pareça o algoritmo ideal por "economizar espaço", na prática ele é o pior em termos de fragmentação. Como ele busca o encaixe mais justo possível (ex: colocar 99 KB num buraco de 100 KB), ele gera um rastro de lacunas minúsculas de 1 KB espalhadas pela RAM. Essas lacunas são inúteis para qualquer outro processo, inutilizando a memória.

5. TRADUÇÃO DE ENDEREÇOS, MMU E SWAPPING

Para garantir a segurança e a abstração da memória em sistemas multitarefa, o processador isola o espaço de endereçamento visto pelo programa da posição física real dos dados:

- **Endereço Lógico (Virtual):** É o endereço gerado pela CPU do ponto de vista do programa (iniciando do endereço 0). O programa não sabe onde está na RAM real.
- **Endereço Físico:** É a localização eletrônica real onde a instrução ou dado reside nos pentes de memória RAM.
- **MMU (Memory Management Unit):** Chip de hardware responsável por traduzir endereços lógicos em endereços físicos em tempo de execução.
- **Swapping:** Mecanismo onde processos inteiros (ou páginas) são movidos da RAM para o disco secundário (*Swap Out*) quando a memória enche, e trazidos de volta (*Swap In*) quando necessários. Quando o processo retorna à RAM, a MMU atualiza suas tabelas para refletir o novo endereço físico sem que o programa perceba.

6. MEMÓRIA VIRTUAL E PAGINAÇÃO

A **Paginação** é a técnica moderna mais importante de gerenciamento de memória. Ela elimina totalmente a exigência de que um programa seja alocado de forma contígua na RAM.

Estrutura da Paginação

O espaço virtual do programa é fatiado em blocos de tamanho fixo chamados **Páginas Virtuais** (ex: 4 KB). A RAM física é dividida em blocos do mesmo tamanho chamados **Molduras de Página (Frames)**.

Tabela de Páginas (Page Table)

Estrutura mantida pelo SO para cada processo. Mapeia qual Página Virtual está associada a qual Moldura Física na RAM, contendo também o **Bit de Presença** (1 na RAM, 0 no Disco).

Esquema de Tradução de Endereço via MMU



O deslocamento (Offset) nunca muda durante a tradução; apenas o número da página vira o número da moldura.

7. O CICLO DA FALTA DE PÁGINA (PAGE FAULT)

Quando a CPU tenta acessar uma página cujo bit de presença na Tabela de Páginas é **0** (significa que a página está no disco e não na RAM), ocorre uma interrupção de hardware chamada **Page Fault**:

1. **Detecção:** A MMU identifica o Bit de Presença igual a 0 e dispara uma interrupção para a CPU.
2. **Interrupção do Processo:** O SO suspende o processo ativo e assume o controle da execução.
3. **Localização no Disco:** O SO busca o endereço da página solicitada na área de *swap* do disco.
4. **Alocação de Moldura:** O SO procura uma Moldura de Página livre na RAM física.
   ** Se a RAM estiver cheia, executa um Algoritmo de Substituição para escolher uma página vítima.*
5. **Carregamento (Swap-In):** A página requerida é lida do disco e copiada para a moldura liberada.
6. **Atualização & Reinício:** A Tabela de Páginas é atualizada (bit vira 1) e o SO reinicia a instrução.

8. ALGORITMOS DE SUBSTITUIÇÃO DE PÁGINAS

Quando ocorre um *Page Fault* e a memória RAM está 100% ocupada, o SO precisa selecionar uma página "vítima" para ser gravada no disco e dar lugar à nova página:

Algoritmo	Regra de Seleção da Página Vítima	Avaliação Prática
FIFO (First-In, First-Out)	Remove a página que está na RAM há mais tempo, independentemente de uso.	Simples, porém ineficiente. Pode remover páginas muito utilizadas.
LRU (Least Recently Used)	Remove a página que **não é acessada há mais tempo** no passado.	Excelente desempenho. Exige suporte de hardware/contadores.
Segunda Chance (Clock)	Variação do FIFO. Usa um bit de uso (R). Se $R=1$, dá uma segunda chance e zera o bit; se $R=0$, remove.	Muito utilizado na prática devido ao excelente custo/benefício.
Ótimo (OPT)	Remove a página que não será utilizada pelo **maior período no futuro**.	Teórico (impossível na prática). Usado apenas como benchmark em simulações.

9. RESUMO COMPARATIVO DE FRAGMENTAÇÃO

Tipo	Onde Ocorre e Causa	Como Resolver
Fragmentação Interna	Dentro do bloco alocado. Ocorre em Partições Fixas e na última página da Paginação.	Reduzir o tamanho do bloco/página de alocação.
Fragmentação Externa	Fora dos blocos. Espaços livres pulverizados na RAM (Partições Variáveis).	Compactação de memória ou adoção de **Paginação**.

Checklist Rápido para a Prova

✓ **Monoprogramativo:** Simples, sem corrida, ocioso em E/S. | ✓ **Best Fit:** Pior em fragmentação na prática.
✓ **MMU:** Traduz Virtual → Físico. | ✓ **Paginação:** Elimina Fragmentação Externa, mantém pequena Fragmentação Interna.